

CSC 250: Foundations of Computer Science I

Spring 2026

Chapter 5: Analysis Tools and Techniques

Sakhawat Hossan

Department of Computer Science
University of North Carolina at Greensboro

Section 5.3 Permutations and Combinations

What is a Permutation? (order counts)

- A *permutation* of a set is an arrangement of the objects from a set.
- There are $n!$ permutations of an n -element set, where $0! = 1$ and $n! = n \cdot (n - 1)!$
- The number of permutations of length r chosen from an n -element set (where $r \leq n$) is

Formula:

$${}_nP_r = \frac{n!}{(n - r)!}$$

Where:

- n = total number of items
- r = number of items chosen
- $n!$ = factorial

Permutation Example

Example:

From 5 students, how many ways can we choose a president, vice president, and secretary?

Since the roles are different, **order matters**.

$${}_5P_3 = \frac{5!}{(5-3)!} = \frac{5!}{2!} = \frac{5 \cdot 4 \cdot 3 \cdot 2!}{2!} = 5 \cdot 4 \cdot 3 = 60$$

Answer: 60 ways

Permutation Example

How many distinct arrangements can be made using all the letters in the word **ALABAMA**?

The word **ALABAMA** has 7 letters total:

- A appears 4 times
- L appears 1 time
- B appears 1 time
- M appears 1 time

Since some letters repeat, we use:

$$\frac{7!}{4!}$$

$$= \frac{5040}{24} = 210$$

Answer: 210 distinct arrangements

What is a Combination?

Combinations (order does not count)

- The number of *combinations of n things taken r at a time* is the number of r -element subsets of an n -element set and is given by the formula.

Formula:

$${}_nC_r = \frac{n!}{r!(n-r)!}$$

Combination Example

Example:

From 5 students, how many ways can we choose 3 students for a group?

Here, the order does **not** matter.

$${}_5C_3 = \frac{5!}{3!(5-3)!} = \frac{5!}{3!2!}$$

$$= \frac{5 \cdot 4 \cdot 3!}{3! \cdot 2 \cdot 1} = \frac{20}{2} = 10$$

Answer: 10 ways

Permutation vs Combination

Permutation

- Order matters
- Arrangement
- Example: choosing class officers

Combination

- Order does not matter
- Selection
- Example: choosing committee members

Quick check:

- ABC, ACB, BAC are different in permutation
- ABC, ACB, BAC are the same choice in combination

Exercise:

1. A class has 8 students. In how many ways can a president, vice president, and secretary be chosen?

Answer: 336

2. A student has 6 different math books. In how many ways can 4 of them be arranged on a shelf?

Answer: 360

3. A student has 6 different math books. In how many ways can they be arranged on a shelf?

Answer: 720

4. There are 10 runners in a race. In how many ways can 1st, 2nd, and 3rd place be awarded?

Answer: 720

Exercise:

1. A club has 9 members. In how many ways can a committee of 4 members be selected?

Answer: 126

2. A restaurant offers 7 different toppings. In how many ways can a customer choose 3 toppings?

Answer: 35

3. An instructor has prepared 12 questions. In how many ways can 5 questions be selected for a quiz?

Answer: 792

Section 5.4 Discrete Probability

Uncertainty refers to the degree of unknown variability or lack of sureness in data, model predictions, or inferences. It differs from the binary notion of "true" or "false" by acknowledging a spectrum of possibilities or degrees of confidence in the outcomes.

A random variable is described as an aspect of the world about which there is uncertainty.

For example,

- To represent possible outcomes when rolling a dice, we can define a random variable Roll, that can take on the values $\{1, 2, 3, 4, 5, 6\}$.
- To represent the status of a flight, we can define a variable Flight that takes on the values $\{\text{on time, delayed, canceled}\}$.

An **Event** is a set of outcomes of an experiment (a subset of the sample space) to which a probability is assigned.

Discrete Probability

Event vs Random Variable: A random variable is a more abstract concept that assigns numbers to outcomes, whereas an event is a specific collection of outcomes.

For example, if X is a random variable representing the roll of a dice, the event $X > 4$ corresponds to the random variable taking on values greater than 4.

Probability is the primary tool for quantifying uncertainty. By assigning probabilities to different events, we effectively express our level of certainty or uncertainty about these events occurring.

Example: $P(\text{rolling an odd number}) = P(\text{rolling a 1}) + P(\text{rolling a 3}) + P(\text{rolling a 5})$

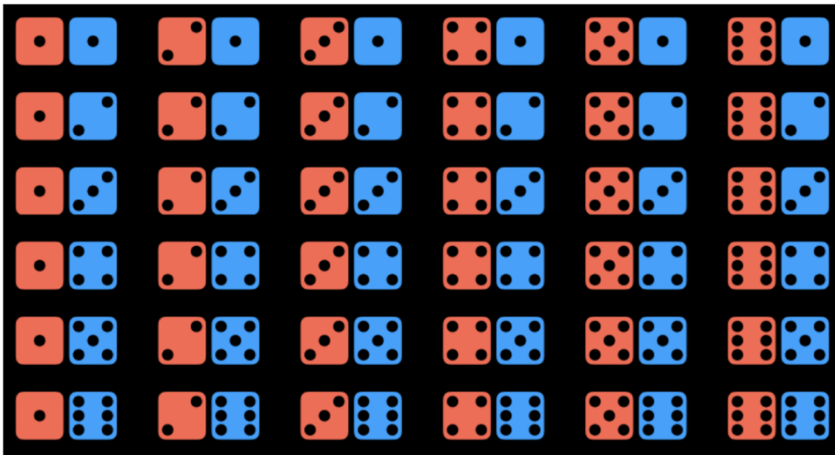
$$= 1/6 + 1/6 + 1/6$$

$$= 1/2$$



Quantify Uncertainty

Consider the event of rolling two dice, what is the probability of the two rolling numbers which sum is 7?



2	3	4	5	6	7
3	4	5	6	7	8
4	5	6	7	8	9
5	6	7	8	9	10
6	7	8	9	10	11
7	8	9	10	11	12

$$P(7) = 1/36 + 1/36 + 1/36 + 1/36 + 1/36 + 1/36 = 1/6$$

Probability Distribution

Often, we are interested in the probability with which each value of the random variable occurs and represent this using a probability distribution. A probability distribution can be represented in the form of a table, which is especially common for discrete random variables.

- If X is a random variable and ω is an event, the probability of ω can be represented as $P(X = \text{a variable value})$ or $P(\omega)$.
- $0 < P(\omega) < 1$: every value representing probability must range between 0 and 1. In general, the higher the value, the more likely the event is to happen.
- The sum of the probabilities of every possible event is equal to 1.

Probability distribution of rolling a die

X	1	2	3	4	5	6	<i>Sum</i>
P(X)	1/6	1/6	1/6	1/6	1/6	1/6	1

Independence

1. Two random variables are independent if the realization of one does not affect the probability distribution of the other.

- Random Variable A : The outcome of the coin toss;
- Random Variable B : The outcome of the die roll .

2. Two events are independent if the occurrence of one does not affect the probability of occurrence of the other or, equivalently, does not affect the odds.

- Event A: Rolling a 6 on the first die.
- Event B: Rolling a 6 on the second die.

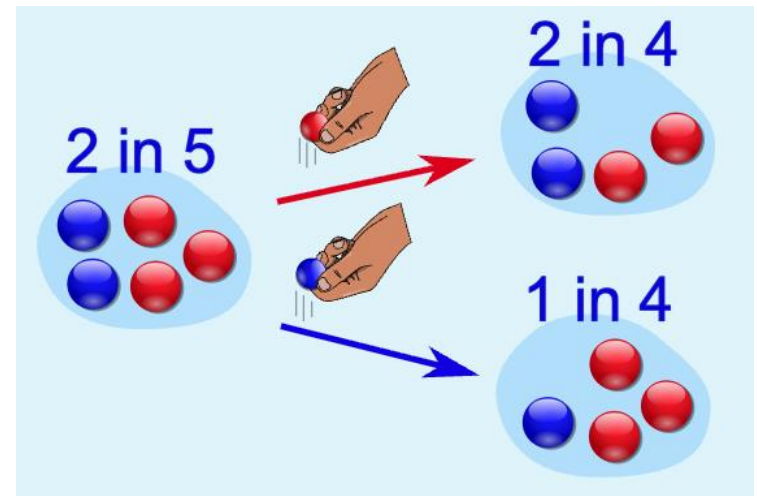
$$P(\text{red die} \text{ and } \text{blue die}) = P(\text{red die})P(\text{blue die}) = \frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36}$$

Dependence

1. Dependent random variables are variables in a probability space where the outcome or realization of one variable affects the probability distribution of another.

- Knowing the value of one variable provides information about the likely values of the other variable.
- This is in contrast to independent random variables, where the outcome of one variable does not influence the outcome of another.

2. Dependent events in probability are events where the outcome or occurrence of one event affects the likelihood of occurrence of another event.



Conditional probability

Conditional probability examines the likelihood of an event occurring, based on the occurrence of a previous event or outcome.

- Denoted as $P(A|B)$, which reads as "the probability of event A occurring given that event B has occurred."
- This concept is crucial when the occurrence of one event affects the likelihood of another event.

Example of dependent events with conditional probability:

- When it's cloudy, there's a 50% chance it will rain.
- If it's not cloudy, the chance of rain drops to 10%.

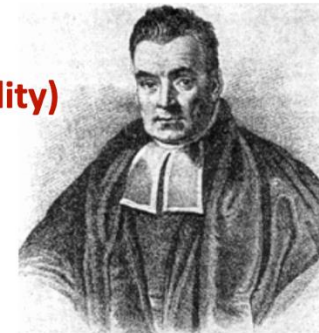
Bayes' theorem

Bayes' theorem (alternatively Bayes' law or Bayes' rule), describes the probability of an event, based on **prior knowledge** of conditions that might be related to the event.

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

Diagram labels for the equation:

- Posterior probability** points to $P(A|B)$
- Likelihood probability** points to $P(B|A)$
- prior (knowledge/probability)** points to $P(A)$
- Marginal likelihood** points to $P(B)$



Bayes, Thomas (1701-1763)

A and B are events, $P(B) \neq 0$

The “**posterior**” conditional probability refers to probabilities obtained *after* the data has been taken into account; whereas the “**prior**” probability is obtained, or posited, *before* our data has been taken into account.

Applying Bayes' Theorem – Example 1



Suppose you hear your dog is barking

Event A: a thief is around

Event B: your dog is barking

$P(A|B)$ is low



How likely is he/she barking at a thief?

Your inference: based on your experience and knowledge, when you hear your dog is barking, you infer that it is unlikely he/she is barking at a thief.



Applying Bayes' Theorem – Example 1

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

A: a thief is around
B: the dog barks

$P(A) = 0.02$ A thief is around on 2% of days.

$P(B) = 0.8$ The dog barks on 80% of days.

$P(B|A) = 0.9$ If a thief is around, the dog barks 90% of the time.

$P(A|B) = ?$ The probability that a thief is around given that the dog barks (your inference)

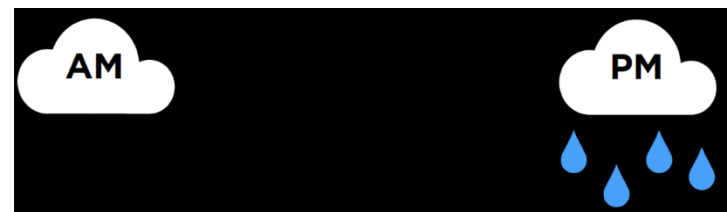
$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)} = \frac{0.9 * 0.02}{0.8} \approx 0.01$$

Applying Bayes' Theorem – Example 2

Given clouds in the morning, what's the probability of rain in the afternoon?

- 80% of rainy afternoons start with cloudy mornings. $P(\text{clouds} | \text{rain}) = 0.8$
- 40% of days have cloudy mornings. $P(\text{clouds}) = 0.4$
- 10% of days have rainy afternoons. $P(\text{rain}) = 0.1$

$$\begin{aligned} &P(\text{rain} | \text{clouds}) \\ &= P(\text{clouds} | \text{rain}) P(\text{rain}) / P(\text{clouds}) \\ &= (0.8)(0.1)/0.4 \\ &= 0.2 \end{aligned}$$



Applying Bayes' Theorem – Example 3

Background: Imagine a hospital where: 90% of people with COVID have a fever. Only 10% of people actually have COVID. 40% of all patients have a fever.

Question: Using Bayes' Theorem, calculate the probability that a patient actually has COVID, given that they have a fever.

Answer: Let, C = patient has COVID and F = patient has a fever

Given:

$$P(F | C) = 0.90$$

$$P(C) = 0.10$$

$$P(F) = 0.40$$

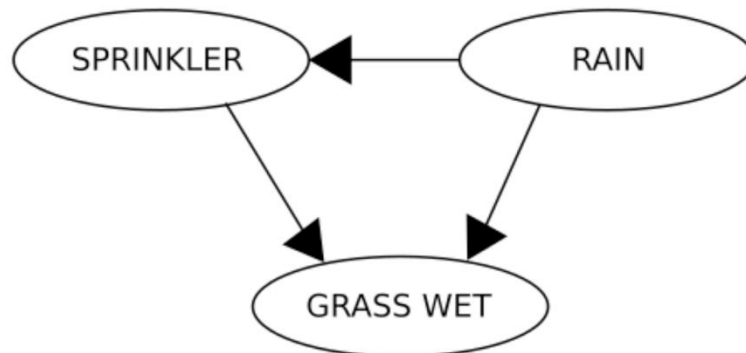
Using Bayes' Theorem:

$$P(C | F) = \frac{P(F | C) P(C)}{P(F)}$$

Bayesian Networks

Bayesian networks are a powerful tool for analyzing and reasoning about complex variable dependency relationships

- A Bayesian network is a directed acyclic graph (DAG).
- Each node on the graph represents a random variable.
- An arrow from X to Y represents that X is a parent of Y. That is, the probability distribution of Y depends on the value of X.
- Each node X has probability distribution.

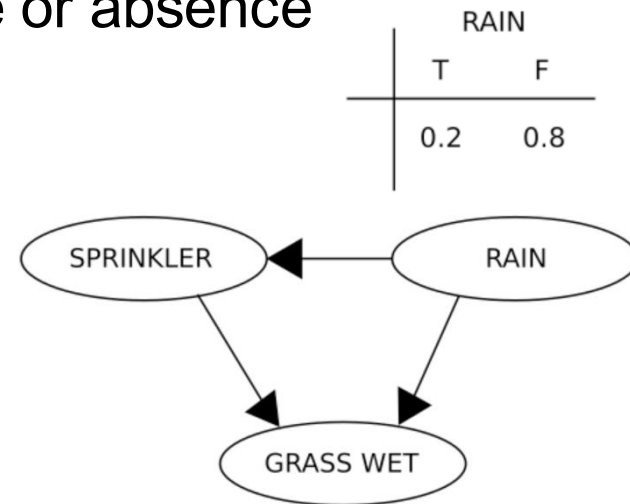


Bayesian Network Example

Suppose we want to model the dependencies between three variables: the sprinkler (or more appropriately, its state - whether it is on or not), the presence or absence of rain and whether the grass is wet or not.

Observe that two events can cause the grass to become wet: an active sprinkler or rain. Rain has a direct effect on the use of the sprinkler (namely that when it rains, the sprinkler usually is not active). This situation can be modeled with a Bayesian network (shown to the right). Each variable has two possible values, T (for true) and F (for false).

RAIN	SPRINKLER	
	T	F
F	0.4	0.6
T	0.01	0.99



SPRINKLER	RAIN	GRASS WET	
		T	F
F	F	0.0	1.0
F	T	0.8	0.2
T	F	0.9	0.1
T	T	0.99	0.01

Represent Bayesian Networks

Natural language :

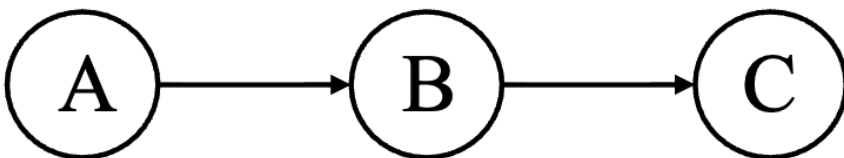
- Description 1: a Bayesian network with three variables: A, B, and C, where A is a parent of B and B is a parent of C.
- Description 2: a Bayesian network with three variables: A, B, and C, where B is dependent on A, and C is dependent on B.

Probability Distribution:

The joint probability distribution of A, B, and C is:

$$P(A,B,C) = P(A) \cdot P(B|A) \cdot P(C|B)$$

Graphical Representation:

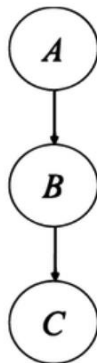


Represent Bayesian Networks

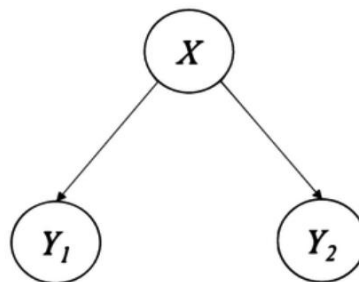
Bayesian Networks have various structural patterns:

1. **Serial (chain, sequential, directed) Structure** : a sequence of nodes where each node is directly connected to the next in a single path.
2. **Diverging (fork, branching) Structure** : a parent node influences two or more child nodes, forming a "fork."
3. **Converging (collider) Structure** : two or more nodes independently influence a common child node. The child node "collides" the paths from its parents.

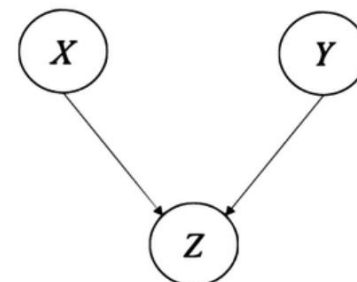
1.



2.



3.



Bayesian Networks Inference Example



whether Tom gets to his appointment on time is affected by the train



Raphael Sanzio, "Take the A Train," 1510

whether the train is on time is affected by its maintenance and weather



Inference: given the weather and train information/knowledge, can we infer how possible Tom will get to his appointment?



whether the maintenance is made is affected by the weather

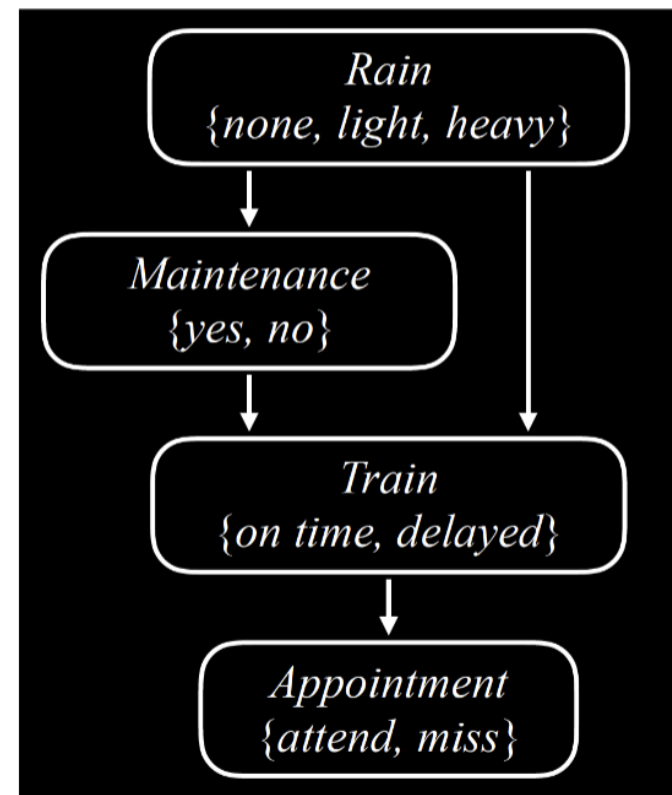
Bayesian Networks Inference Example

Natural Language :

there are four variables and their dependency relations are:

1. whether the maintenance is made is affected by the weather.
2. whether the train is on time is affected by its maintenance and weather.
3. whether Tom gets to his appointment on time is affected by the train.

Bayesian Network Graph



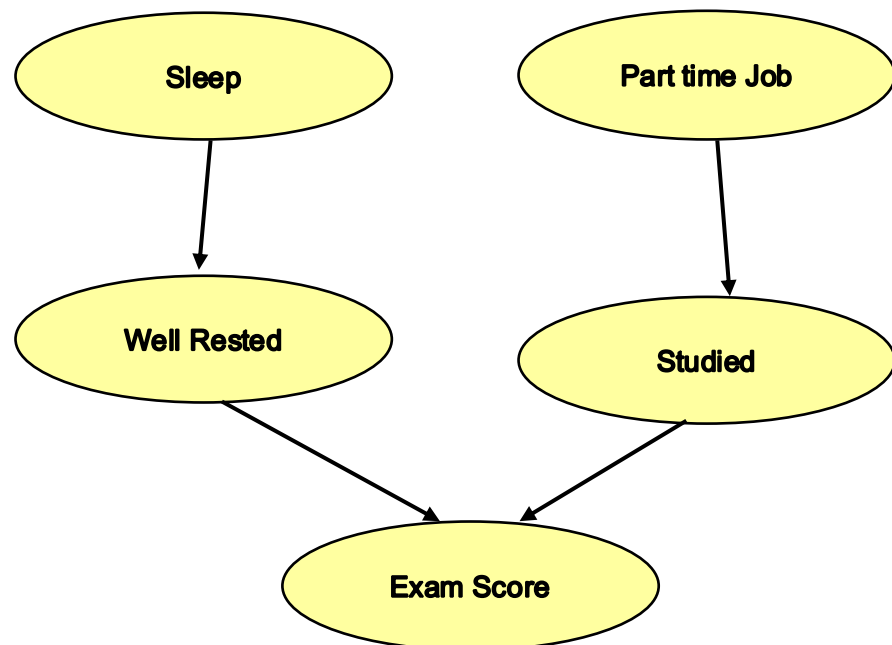
Bayesian Networks Inference Example 2

Topic: Student performance

A student's exam score depends on how much they studied and whether they were well rested. Being well rested depends on whether the student got enough sleep. The amount of study time may depend on whether the student had a part-time job, since working more hours can reduce study time.

Random Variables:

- Part-time Job
- Studied
- Sleep
- Well Rested
- Exam Score



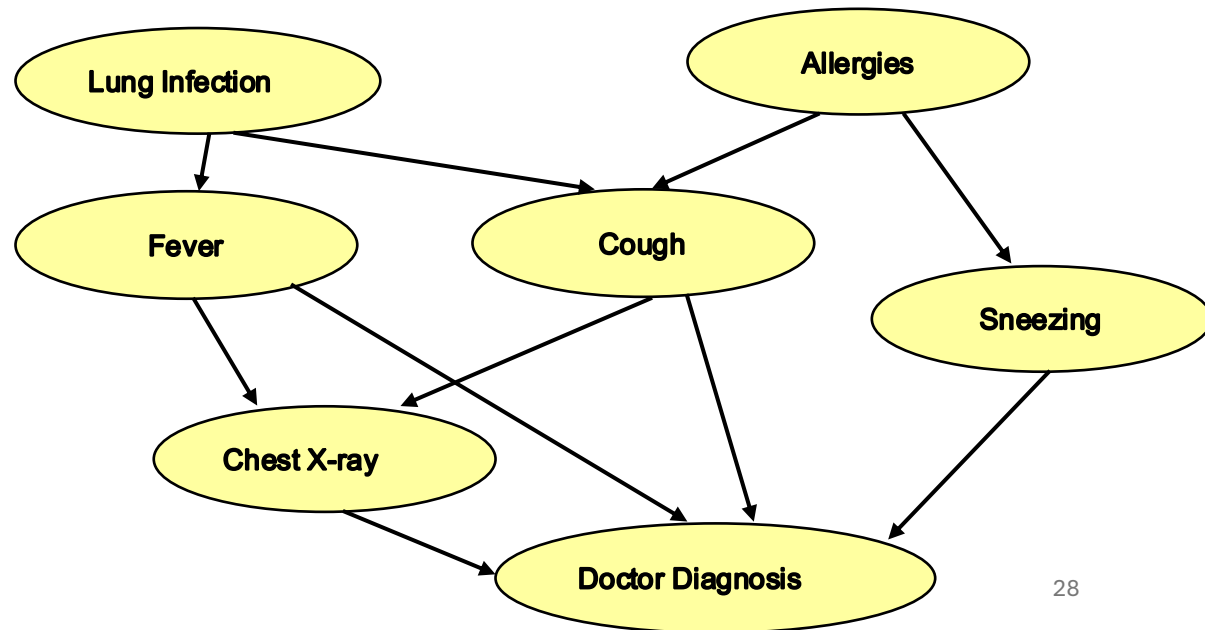
Bayesian Networks Inference Example 3

Topic: Medical diagnosis

A patient may have a lung infection or allergies. A lung infection can cause fever and cough. Allergies can cause cough and sneezing. A positive chest X-ray is more likely if the patient has a lung infection. A doctor's diagnosis depends on the patient's fever, cough, sneezing, and X-ray result.

Random Variables:

- Lung Infection
- Allergies
- Fever
- Cough
- Sneezing
- Chest X-ray
- Doctor Diagnosis



Transition Matrix

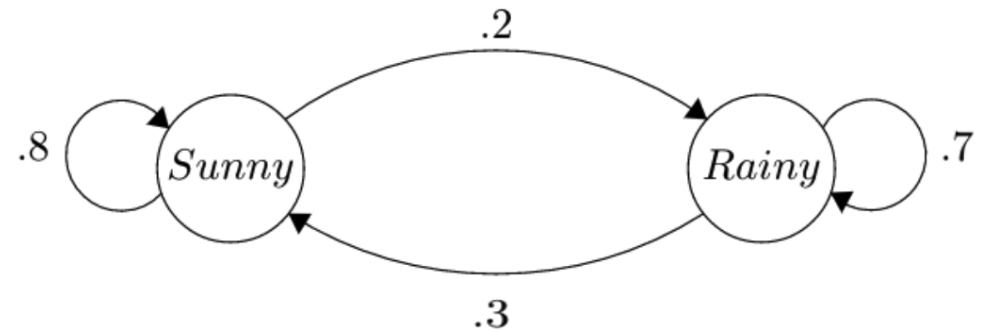
- The Transition Matrix is a square matrix that describes the probabilities of moving from one state to another.
- The Transition Matrix can be visualized as a directed graph where nodes represent states and directed edges represent transitions between states.

		Tomorrow (X_{t+1})	
Today (X_t)		0.8	0.2
		0.3	0.7

Transition Matrix Example

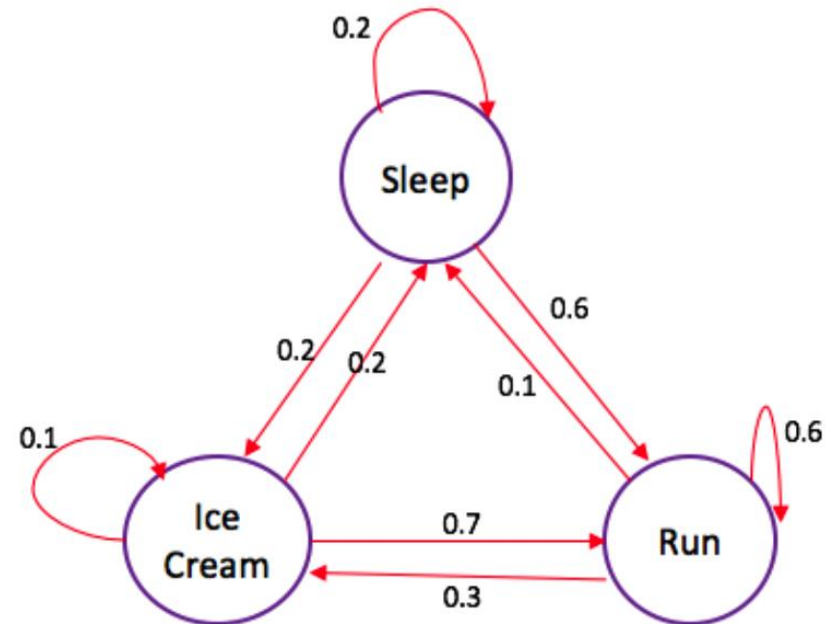
- If today is Sunny, there is a 0.8 probability that tomorrow will also be Sunny and a 0.2 probability that it will become Rainy. If today is Rainy, there is a 0.7 probability that tomorrow will remain Rainy and a 0.3 probability that it will become Sunny.
- The corresponding transition matrix is:

		Tomorrow (X_{t+1})	
			
Today (X_t)		0.8	0.2
		0.3	0.7

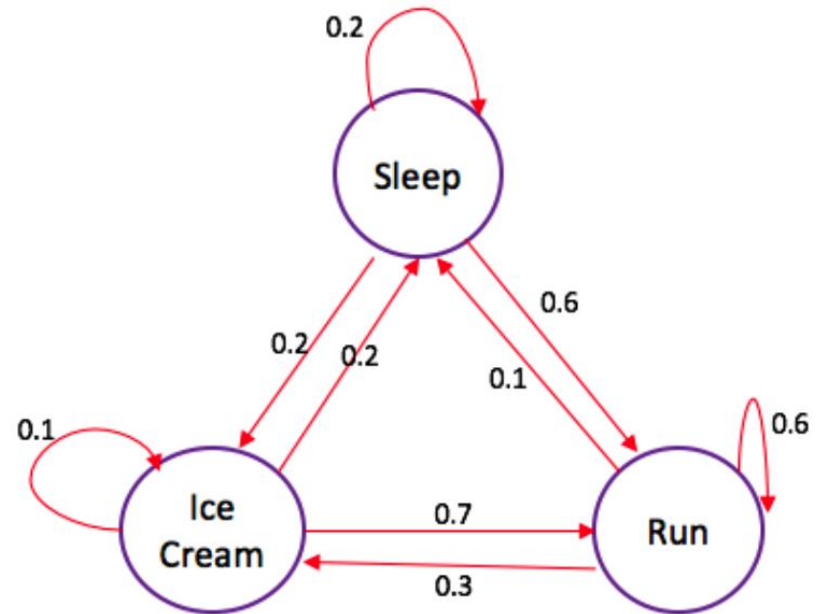


Transition Matrix Example 2

- This diagram shows a Markov chain with three possible states: **Sleep**, **Ice Cream**, and **Run**. Each directed arrow represents the probability of moving from the current state to the next state in one step, and each loop shows the probability of staying in the same state. Thus, the graph visually represents how the system changes over time from one activity to another according to the given transition probabilities.



Transition Matrix Example 2



CURRENT STATE	NEXT STATE			
		SLEEP	RUN	ICE CREAM
	SLEEP	0.2	0.6	0.2
	RUN	0.1	0.6	0.3
	ICE CREAM	0.2	0.7	0.1

Section 5.1 Analyzing Algorithms

What is Algorithm Analysis?

Algorithm analysis studies how much **time** and **memory** an algorithm needs as the input size grows. The main goal is to compare algorithms in a way that is less dependent on a specific machine or programming language.

Algorithm Analysis Focus on:

- Running time
- Space usage
- Growth as input size increases

Why Do We Need Algorithm Analysis?

Two algorithms may solve the same problem, but one may require much more work than the other.

Why this matters:

- Small inputs may not show much difference
- Large inputs can make inefficient algorithms very slow
- Good analysis helps us choose better solutions

Example:

Searching for a value in a list of 5 items is easy.

Searching for a value in a list of 1,000,000 items is very different.

What Do We Measure?

In algorithm analysis, we are often interested in:

1. Running time

How many steps or operations are performed?

2. Memory usage

How much extra storage is needed?

For this lecture, we mainly focus on **running time**.

Important:

We usually do not count actual seconds.

Instead, we count the amount of work done.

RAM Model

RAM model stands for **Random Access Machine**.

It is a simplified way to think about how a computer runs an algorithm.

Assumptions of the RAM model:

- There is a **single processor**
- Instructions are executed **one at a time**
- Basic operations take about the same amount of time
- Memory can be accessed directly

Why we use it:

It gives us a simple standard model for studying algorithms without worrying about machine-specific details.

Example Problem – Linear Search

Problem:

Given a list of values and a target value, determine whether the target appears in the list.

Example list:

4,9,2,7,12,5

Target:

7

Linear search idea:

Start from the first element and compare one by one until:
the target is found, or the list ends .

Linear Search Walkthrough

Search for **7** in:

4,9,2,7,12,5

Step-by-step:

- Compare 7 with 4 → not equal
- Compare 7 with 9 → not equal
- Compare 7 with 2 → not equal
- Compare 7 with 7 → found
- So, the algorithm stops after **4 comparisons**.

Point:

The number of comparisons depends on where the target appears.

More Detailed Linear Search Example

Now search for **5** in:

4,9,2,7,12,5

Step-by-step:

- Compare 5 with 4
- Compare 5 with 9
- Compare 5 with 2
- Compare 5 with 7
- Compare 5 with 12
- Compare 5 with 5 → found
- This time, it takes **6 comparisons**.

Observation:

If the item is near the end, more work is needed.

What If the Target Is Not in the List?

Search for **10** in:

4,9,2,7,12,5

The algorithm must check:

4 9 2 7 12 5

Since 10 is not in the list, the algorithm checks **every element**.

Conclusion:

Sometimes the algorithm finishes early, and sometimes it must examine the entire input. This is why algorithm analysis studies different situations.

Calculating Running Time of Linear Search

Goal: count how many comparisons the algorithm makes.

- Suppose the list has n elements.

Case 1: Best case

The target is in the **first position**. Only **1 comparison** is needed.

Case 2: Worst case

The target is in the **last position**, or it is **not in the list**. The algorithm checks all n elements.

Case 3: Average idea

If the target could be anywhere, the algorithm may check about half of the list on average ($n/2$)

Calculating Running Time of Linear Search

Conclusion:

- Best case: 1 comparison
- Worst case: n comparisons
- Average case: about $n/2$ comparisons

Main point:

As the list gets larger, the running time grows **linearly** with the input size.

Space Complexity of Linear Search

Space complexity tells us how much extra memory an algorithm uses.

For linear search, we usually only need:

- the list itself
- the target value
- one index or counter variable

The algorithm does **not** create another large list or extra structure. So, the extra memory used stays the same, even if the input list becomes very large.

Conclusion:

- Linear search uses **constant extra space**
- Space complexity: $O(1)$

Another Search Algorithm – Binary Search

Binary search works on a **sorted list**. Instead of checking one element at a time, it looks at the **middle element** and eliminates half of the list each step.

Example: Search for 23 in

3,7,12,18,23,31,45,56,70

Steps:

1. Check middle element 23
2. Found immediately

Another example, search for 31:

1. Check middle: 23
2. Since $31 > 23$, search the right half
3. Check middle of right half: 56

Another Search Algorithm – Binary Search

4. Since $31 < 56$, search the left part
5. Check 31 → found

Main idea:

Each step cuts the search space in half.

Running Time of Binary Search

Suppose the list has n elements.

Because binary search removes half of the remaining list each step:

- after 1 step: $n/2$
- after 2 steps: $n/4$
- after 3 steps: $n/8$

So, the number of steps grows much more slowly than linear search.

Running time:

- Best case: **1 comparison**
- Worst case: $\log_2 n$ comparisons
- Average case: also grows about $\log_2 n$

n	1	2	4	8	16	32
$\log n$	0	1	2	3	4	5

Conclusion:

Binary search is much faster than linear search for large sorted lists.

Space Complexity of Binary Search

If binary search is written **iteratively**, it only uses a few variables:

- low
- high
- middle

So, the extra memory stays fixed.

Iterative binary search:

- Space complexity: $O(1)$

If binary search is written **recursively**, each recursive call uses memory on the call stack.

Recursive binary search:

- Space complexity: $O(\log n)$

Linear Search vs Binary Search

Key lesson:

Linear search is simpler and works on any list.

Binary search is faster, but the list must already be sorted.

Algorithm	List Requirement	Best Time	Worst Time	Extra Space
Linear Search	list can be unordered	1	(n)	$O(1)$
Binary Search	list must be sorted	1	(log n)	$O(1)$ iterative

Asymptotic Notations

When we analyze algorithms, exact running time can depend on many things:

- computer speed
- programming language
- compiler
- operating system
- input data

Instead of counting exact seconds, we study how the algorithm grows as the input size becomes large.

Asymptotic notation helps us describe the growth rate of an algorithm.

Input Size and Running Time

Let n represent the size of the input.

Examples:

- If we process a list, n = number of elements
- If we process a string, n = number of characters
- If we process a graph, n may be number of vertices or edges

We often write running time as a function:

$$T(n)$$

This means the amount of work the algorithm does for input size n .

Example of a Running Time Function

Suppose an algorithm has this running time:

$$T(n) = 3n + 5$$

This means the algorithm does about:

- $3n$ repeated operations
- plus 5 extra fixed operations

If n becomes very large, the $3n$ part matters much more than the 5. So, we focus on the **dominant term**.

Dominant Term

The **dominant term** is the part of the function that grows fastest as n increases.

Examples:

$$T(n) = 3n + 5$$

Dominant term:

$$T(n) = 2n^2 + 10n + 20$$

Dominant term:

$$T(n) = 7n^3 + 2n^2 + 100$$

Dominant term:

$$n^3$$

For large n , the fastest-growing term controls the behavior.

Ignoring Constants

In asymptotic analysis, we usually ignore constant multipliers.

Examples:

$$T(n) = 5n$$

and

$$T(n) = 100n$$

Both grow linearly with n .

So, both are described as:

$$O(n)$$

Even though $100n$ is larger than $5n$, they have the same general growth pattern.

Big-O Notation

Big-O describes an **upper bound** on an algorithm's growth.

It tells us the algorithm does **no more than** a certain amount of work, up to constant factors.

Common meaning:

$O(\text{something})$

means the algorithm grows **at most** like that function.

Examples:

- $O(1)$:constant time
- $O(n)$:linear time
- $O(n^2)$:quadratic time
- $O(\log n)$:logarithmic time

Big-O Examples

Simplify each running time using Big-O.

Example 1

$$T(n) = 4n + 10$$

Dominant term is n .

$$T(n) = O(n)$$

Example 2

$$T(n) = 6n^2 + 3n + 9$$

Dominant term is n^2 .

$$T(n) = O(n^2)$$

Example 3

$$T(n) = 20$$

The running time does not grow with n .

$$T(n) = O(1)$$

Big-Omega Notation

Big-Omega is written as:

Ω

Big-Omega describes a **lower bound**.

It tells us the algorithm takes **at least** a certain amount of work.

Example:

If an algorithm must look at every element in a list at least once, then the running time is:

$\Omega(n)$

That means the algorithm cannot do better than linear work for that problem.

Big-Theta Notation

Big-Theta is written as:

$$\Theta$$

Big-Theta means the running time is bounded from both above and below by the same growth rate.

So:

$$T(n) = \Theta(n)$$

means the algorithm grows exactly linearly, ignoring constants.

Another way to say it:

$$\Theta(g(n))$$

means the running time grows at the same rate as $g(n)$.

Big-O vs Big-Omega vs Big-Theta

Notation	Meaning	Simple idea
O	Upper bound	At most
Ω	Lower bound	At least
Θ	Tight bound	Same growth rate

Example:

If

$$T(n) = 5n + 3$$

then:

$$T(n) = O(n)$$

$$T(n) = \Omega(n)$$

$$T(n) = \Theta(n)$$

because the function grows linearly.

Common Growth Rates

From smaller growth to larger growth:

- $O(1)$
- $O(\log n)$
- $O(n)$
- $O(n \log n)$
- $O(n^2)$
- $O(n^3)$
- $O(2^n)$

Important idea:

As n becomes large, slower-growing functions are usually better.

Algorithm Example 1 — Finding the Maximum

Problem:

Given a list of numbers, find the largest number.

Example list:

[8, 3, 12, 5, 9]

Basic idea:

- Assume the first number is the maximum.
- Compare it with the next number.
- If the next number is larger, update the maximum.
- Continue until the end of the list.

Algorithm Example 1 — Finding the Maximum

Example list:

[8, 3, 12, 5, 9]

For the list above:

- Start with 8 as max
- Compare 3 with 8 → max stays 8
- Compare 12 with 8 → max becomes 12
- Compare 5 with 12 → max stays 12
- Compare 9 with 12 → max stays 12

Final answer: 12

Analysis of Finding the Maximum

Suppose the list has n elements.

The algorithm must check each element to make sure no larger value exists. Number of comparisons is about:

$$n - 1$$

So, the running time grows linearly.

$$T(n) = n - 1$$

Ignoring constants and smaller terms:

$$T(n) = \Theta(n)$$

Time complexity:

$$O(n)$$

Space complexity:

$$O(1)$$

because we only store one current maximum value.

Algorithm Example 2 — Selection Sort

Problem:

Sort a list of numbers from smallest to largest.

Example list:

[6, 2, 8, 4]

Selection sort idea:

- Find the smallest value in the list.
- Put it in the first position.
- Find the next smallest value.
- Put it in the second position.
- Continue until the list is sorted.

Algorithm Example 2 — Selection Sort

Step-by-step:

Original:

[6, 2, 8, 4]

Smallest is 2, swap with 6:

[2, 6, 8, 4]

Next smallest in remaining part is 4:

[2, 4, 8, 6]

Next smallest is 6:

[2, 4, 6, 8]

Sorted list:

[2, 4, 6, 8]

Analysis of Selection Sort

For selection sort, we repeatedly search for the smallest element.

For n elements:

- first pass checks about n elements
- second pass checks about $n - 1$ elements
- third pass checks about $n - 2$ elements
- and so on

Total work:

$$n + (n - 1) + (n - 2) + \cdots + 1$$

This sum is about:

$$\frac{n(n + 1)}{2}$$

which grows like:

$$n^2$$

Analysis of Selection Sort

So selection sort has:

$$T(n) = \Theta(n^2)$$

Time complexity:

$$O(n^2)$$

Space complexity:

$$O(1)$$

because it sorts mostly in place and does not need a second large list.